Software Design Analysis Report

Project: Smart Hospital Management System (SHMS)

A. Project Details

Field	Details
Project title	Smart Hospital Management System (SHMS)
Team / roll numbers	Harsh Vardhan - 2400290120110 Pallavi Tiwari - 2400290110199 Jahnavi Pandey - 2400290120122 Ishika Garg - 2400290120121 Harshit Kumar Singh - 2400290120112
Project objective	SHMS is a web-based platform that digitises hospital operations: patient registration, appointments, doctor and staff management, billing, pharmacy, laboratory reports and automated alerts. It reduces paperwork and waiting time, and gives doctors quick access to a patient's complete medical history.
Technology stack	Frontend: React.js, HTML/CSS. Backend: Node.js with Express (REST APIs). Database: PostgreSQL. Security: JWT authentication, bcrypt password hashing, role-based access control. Notifications: SMTP email and SMS gateway. Tools: Git/GitHub, Postman, StarUML/draw.io for design.

B. Module Identification and Modularization

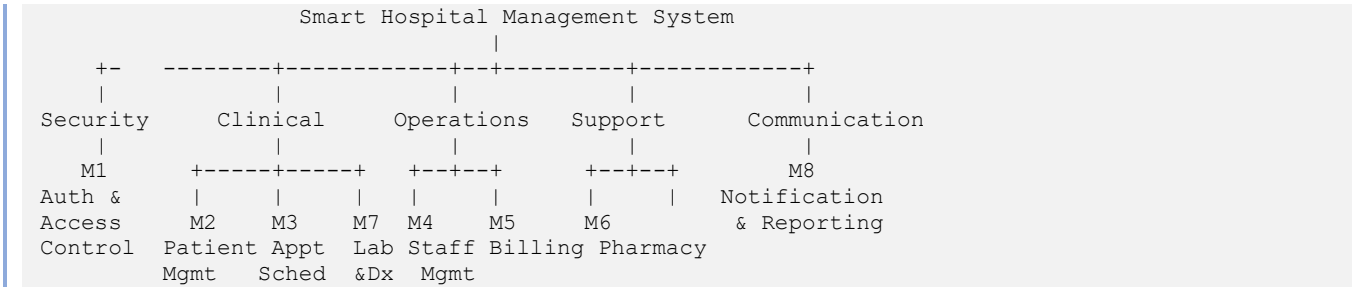
Total number of modules: 8

The system is divided by business function. Each hospital department (reception, doctors, billing, pharmacy, laboratory) maps to one module, so each module has one clear owner and one reason to change. Security (authentication) and communication (notifications) are separated because every other module uses them. This division allows team members to work in parallel, test modules independently, and change one department's rules without affecting the others.

ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Authentication & Access Control	Login, logout, session tokens, role permissions (Admin, Doctor, Nurse, Receptionist, Pharmacist, Lab Technician, Patient)	In: username, password. Out: JWT token, role	None
M2	Patient Management	Register patients, store and update demographics and medical history	In: patient details. Out: patient ID, patient record	M1
M3	Appointment Scheduling	Book, reschedule and cancel appointments; check doctor availability	In: patient ID, doctor ID, date/time. Out: appointment confirmation	M1, M2, M4
M4	Doctor & Staff Management	Maintain staff profiles, departments, specialisations and duty schedules	In: staff details, shift data. Out: staff list, availability slots	M1
M5	Billing & Payment	Generate invoices for consultation, lab and pharmacy charges; record payments	In: service charges, payment details. Out: invoice, receipt	M2, M3, M6, M7
M6	Pharmacy & Inventory	Manage prescriptions, dispense medicines, track stock and low-stock levels	In: prescription, medicine data. Out: dispensed items, stock status	M1, M2
M7	Laboratory & Diagnostics	Record test orders, store results and generate lab reports	In: test order, result values. Out: lab report	M1, M2

ID	Module name	Main responsibility	Inputs / outputs	Depends on
M8	Notification & Reporting	Send appointment reminders, report-ready and low-stock alerts; produce admin analytics reports	In: event triggers from other modules. Out: email/SMS, summary reports	M2, M3, M6, M7

Module hierarchy / decomposition diagram:



C. Design Strategy

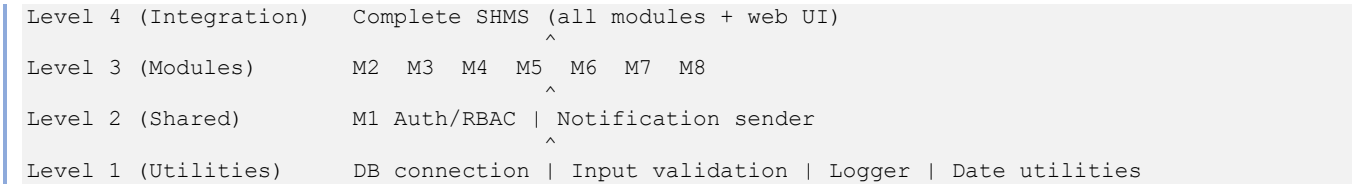
Selected strategy: ☐ Top-Down ☐ Bottom-Up ☒ Hybrid

Justification: A hospital system has clear high-level departments that are best identified top-down, so we first defined the whole system and split it into modules. However, some components such as authentication, database access, validation and notification sending are needed by many modules and are best built first as reusable bottom-up components. A hybrid strategy lets us plan the architecture from the top while building shared services from the bottom. This reduces duplicated code and keeps the final integration smooth. It also lets the team start coding the common services while the detailed module design is still being refined.

Evidence:

Top-down step: We started with the whole system ("manage hospital operations digitally"), divided it into five functional areas (security, clinical, operations, support, communication), and then into the eight modules M1–M8 shown in Section B. Each module was then broken into functions (for example, M3 into check availability, book, reschedule, cancel).

Bottom-up step: Reusable components were built first and then integrated into the modules:



D. Design Approach

Selected approach: ☐ Function-Oriented ☒ Object-Oriented ☐ Mixed

Justification: A hospital naturally consists of real-world entities such as patients, doctors, appointments, prescriptions, lab tests and bills. Each has its own data and behaviour, which maps directly to classes and objects. Object-oriented design allows encapsulation of sensitive patient data, inheritance for different user roles (Doctor, Nurse and Receptionist all extend User), and polymorphism for different payment methods. It also makes the system easier to extend, for example by adding a new department later.

Main classes: User, Patient, Doctor, Nurse, Admin, Receptionist, LaboratoryStaff, Pharmacist, Appointment, MedicalRecord, Prescription, LabTest, Bill, Payment, Medicine, Report (16 classes, modelled in StarUML).

Evidence (class structure chart):

```
                    User
        (userId, username, role, login())
                      ^
   +-------+------+----+-----+------+--------+----------+
 Patient Doctor Nurse Admin Receptionist LaboratoryStaff Pharmacist

Patient 1 ------- * Appointment * ------- 1 Doctor
Patient 1 ------- * LabTest * ------------ 1 LaboratoryStaff
Doctor  1 ------- * Prescription * ------- * Medicine
Patient 1 ------- * Bill 1 <>------------- * Payment
```

Explanation: Inheritance is used for the user types (all role classes extend User). Associations show that one patient can have many appointments, tests and bills. Each class contains its own methods (for example, Appointment.bookAppointment(), Bill.calculateTotal()), so responsibilities stay inside the correct class. The full class diagram is provided in the attached StarUML model.

E. Coupling Analysis

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M3 Appointment ↔ M4 Staff	M3 asks M4 for a doctor's available slots by passing doctorId and date, and receives a list of free slots	Data coupling (low, desirable): only simple parameters are exchanged	Already low. Keep a defined interface such as getAvailableSlots(doctorId, date)
M5 Billing ↔ M6 Pharmacy	M5 receives the list of dispensed items and prices to add to the invoice	Data coupling (low)	Pass only a simple data object (itemId, quantity, price); do not give M5 access to pharmacy tables
M2 Patient ↔ M1 Auth	M2 checks the user's token and role before any record is viewed or changed	Stamp coupling (medium): a whole token/user object is passed although only role and ID are needed	Pass only userId and role, or expose hasPermission(userId, action)
M8 Notification ↔ M3 / M6 / M7	Other modules trigger alerts (reminders, report ready, low stock)	Control coupling (medium) if modules pass flags telling M8 what to do; low if event-based	Use an event/message approach: modules publish an event (AppointmentBooked) and M8 decides how to notify
M7 Laboratory ↔ M2 Patient	M7 reads patient ID and basic details and attaches the report to the patient history	Data coupling (low)	Use patient ID only; access records through M2's API, not directly from the database

Overall: most interactions use data coupling. No module uses content coupling or common (global) coupling, because the database is accessed only through each module's own service layer.

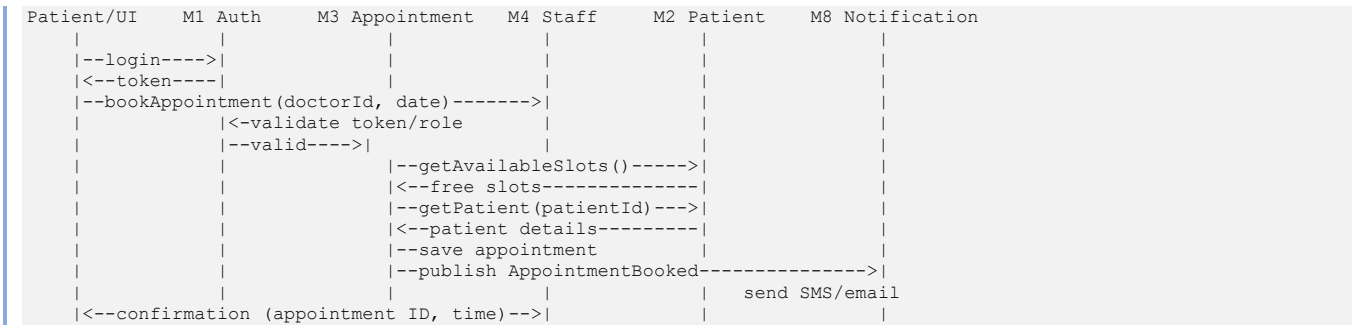
F. Cohesion Analysis

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 Authentication & Access Control	Verify identity and enforce role permissions	Functional cohesion (highest): all tasks (login, token creation, permission check) serve one purpose	None needed
M3 Appointment Scheduling	Manage the appointment lifecycle: book, reschedule, cancel, check conflicts	Functional cohesion: every function works toward managing appointments	Keep reminder sending in M8 so M3 stays focused
M5 Billing & Payment	Generate invoices and record payments	Functional / sequential cohesion: the calculated invoice feeds into payment recording	Separate report-generation functions into M8 to avoid mixing in unrelated tasks
M6 Pharmacy & Inventory	Dispense prescribed medicines and maintain stock	Sequential cohesion (high): dispensing output updates the stock	Move the low-stock alert sending to M8
M8 Notification & Reporting	Send alerts and produce reports	Logical cohesion (low–medium): two related but distinct jobs	Split into two sub-components: Notification Service and Report Generator

Overall: most modules have functional or sequential cohesion, which is high. M8 is the weakest and is improved by splitting it internally.

G. Module Interaction and Design Quality

Sequence diagram: patient books an appointment and receives a confirmation



Design quality summary:

- **Modularity:** eight modules, each with one clear purpose.
- **Coupling:** low; interactions are mostly data coupling through defined interfaces, and notifications are event-driven.
- **Cohesion:** high; most modules are functionally cohesive, and the weakest module (M8) has an improvement plan.
- **Security and privacy:** all access passes through M1, protecting sensitive medical data.
- **Maintainability and scalability:** a new department (for example, Radiology or Telemedicine) can be added as a new module without changing the existing ones.